KNU 강원대학교
KANGWON NATIONAL UNIVERSITY

R E S U L T R E P O R T

마이크로프로세서응용및실험 결과보고서_PWM



수 강 과 목	: 마이크로프로세서응용및실험-2분반
담 당 교 수	: 김도인 교수님
학 번	: 202210820
작 성 자	: 이동제
공 동 실 험 자	: 강민수(202214221)
제 출 일	: 2024-05-30 목요일

1. 실험목적

PWM(Pulse Width Modulation)의 발생 방법을 익힌다.

2. 실험원리

타이머/카운터0과 2만 사용합니다. 그중에서 8-비트 Timer/Counter0을 중점적으로 사용합니다.
그리고 타이머/카운터의 기본 동작 중 PWM 파형 발생에 대해 실험합니다.

PWM이란 Pulse Width Modulation의 약자로 펄스 폭 변조 기능을 뜻합니다.
이름 그대로 펄스 폭을 변형시킬 수 있는 파형으로 파형의 듀티비를 조절하여 변형시킬 수 있습니다.

그중 파형이 출력되는 OC0과 OC2핀은 PB4와 PB7에서 사용됩니다.

TCCR0 - 고속 및 위상 교정 PWM 모드 선택. OCn의 출력 형태 지정(반전 or 비반전), 입력 클럭 분주비(프리스케일러) 지정을 선택할 수 있음.비교 매치 동작일 때 OC0 핀의 동작을 선택할 수 있음.
OCR0 - 항상 카운터 TCNT0의 값과 비교되는 8비트 데이터가 저장되어 있음.
TCNT0 - OCRn과 비교되는 값으로 사용되는 카운터 레지스터로, 값이 같을 시 인터럽트 발생.
TIMSK - 인터럽트 발생을 오버플로우 인터럽트와 출력 비교매치 인터럽트 중 선택하여 처리할 수 있음.
TIFR - 타이머/카운터의 인터럽트를 발생 여부를 알려줌. TCNTn 값과 OCRn 값이 일치하면 OCFn=1이 됨

비교매치 인터럽트(Compare Match Event Interrupt): TCNT0의 값과 OCR0의 값이 일치할 경우 인터럽트 발생합니다.
PWM 주기는 TCNT0의 값으로, 펄스폭은 OCR0의 값으로 설정됩니다.
PWM 파형은 핀 OC0(PB4)를 통해 출력할 수 있습니다.

제작한 ATmega128의 LED는 0일 때 on되는 low active 동작을 하기 때문에 유의해야 합니다.

PWM 모드의 종류	
fast PWM mode (고속 PWM 모드)	Phase Correct PWM mode(위상 교정 PWM 모드)
- 단일 경사 동작 - 카운터 값이 최대값 255(0xff)와 같아질 때마다 타이머/카운터 오버플로 플래그가 셋되고, 인터럽트가 발생함.	- 이중 경사 동작 - 카운터는 0(0x00)에서 최대값 2^8개(=255, 0xff)까지 카운트하고 최대값까지 카운트하면 방향을 역으로 바꾸어 0(0x00)까지 카운트함. - 동작 주파수는 Fast PWM mode의 1/2배임. - ex) fast PWM : f=60Hz -> PC PWM : f=30Hz

표 1) PWM 모드의 종류와 동작 방식

이러한 실험 원리들을 가지고 PWM 기능을 통해 AVR studio에서 작성한 코드를 가지고 오실로스코프 화면에 펄스 파형을 출력하는 테스트를 진행합니다.

3. 실험기구 및 재료

실험기구 및 재료	수량
ATmega128 마이크로 컨트롤러	1개
컴퓨터	1개
ISP 케이블	1개
프로브 케이블	1개
오실로스코프	1개

표 2) 실험기구 및 재료

4. 실험방법

PWM 기능을 사용하는 테스트를 진행하기 위해 AVR Studio에서 코드를 작성 후 오실로스코프를 통해 측정합니다.

이번 실습교재9의 예제 1번과 2번을 통해 파형 출력 모드를 fast PWM mode와 Phase Current PWM mode로 변환하여 출력하는 코드를 작성하고 오실로스코프를 통해 확인합니다. 차이점은 둘이 비교 일치가 발생하는 위치가 다르기 때문에 서로 다른 주파수에서 작동한다는 것입니다.
예제 3번은 Timer/Counter1을 사용하는데 이번 실습에서는 Timer/Counter0과 2만 사용한다고 가정했기 때문에 테스트하지 않았습니다.

후에 구술할 내용은 추가 실습에 관해 작성하겠습니다.

1. 외부 인터럽트를 이용하여 듀티비 조절하시오. (Timer/Counter0 fast PWM mode)

먼저 오실로스코프에 연결하기 전 파동을 출력하기 위한 코드를 작성합니다.

```
#include<avr/io.h>
#include<avr/interrupt.h>

int a = 127;
```

먼저 I/O 및 인터럽트 관련 함수를 사용하기 위해 헤더 파일을 포함한다는 선언을 합니다.
그리고 PWM의 듀티 사이클을 조절하기 위한 a라는 변수를 127로 지정하고 사용하겠다고 해놓고 시작합니다.

```
ISR(INT0_vect)
{
        a = a-63;
}
```

```
ISR(INT1_vect)
{
        a = a+63;
}
```

ATmega128에 장착된 스위치. 그러니깐 즉 외부 인터럽트 발생으로 인한 인터럽트 함수를 사용하기 위해 INT0_vect와 INT1_vect를 대입하여 사용합니다. 그리고 스위치0을 누를땐 a가 63씩 감소하고, 스위치1을 누를 땐 a가 63씩 증가하도록 만듭니다.

```
int main(void)
{
        SREG |= 0x80;
        EIMSK=0x03;
        EICRA=0x0F;
```

SREG | 비트로 전역 인터럽트를 활성화하고, EIMSK를 통해 방금 위에서 인터럽트 함수를 쓰기 위해 대입했던 INT0과 INT1의 인터럽트를 허용합니다. 그리고 EICRA를 통해 인터럽트가 모두 상승 엣지에서 동작하도록 합니다.

```
        DDRB=0xff;      // OC0 출력설정
        PORTB=0xff;      // LED off
        TCCR0=0x6f;     // f=61.0351Hz, 분주비 1024, fast PWM mode
                        // clear OC0 on compare Match, set OC0 at BOTTOM
```

DDRB 함수를 통해 PORTB를 출력으로 설정하고, 그중 OC0 핀을 PWM 출력을 할 수 있는 핀으로 설정합니다. 그리고 PORTB에 0xff를 대입해 값을 모두 1을 넣어주어 LED를 꺼줍니다. TCCR0을 통해 모드를 단일 경사 모드인 Fast PWM으로 설정하고, 비교 매치 동작을 하도록 만들고 작동 후에 OC0이 시작점으로 셋되게 만듭니다. 그리고 분주비를 1024로 만들어 f_{OCn} = f_{clkTn}/2*N*(1+OCRn)이라는 식을 통해서 16MHz/2*1024*(1+127)로 계산됩니다. 주파수를 61.0351Hz가 될 수 있도록 만듭니다.
. 그리고 외부 인터럽트 발생에 의해 OCRn=127이란 값이 ±63씩 변동될 수 있습니다.

```
        TCNT0=0x00;

        while(1)
        {
                OCR0=a;
                 };

}       // do nothing
```

TCNT0으로 타이머/카운터0의 레지스터의 값을 0으로 초기화합니다. 그리고 while문 안에 비교 일치 레지스터 OCR0=a;를 넣어 한 번이 아닌 무한 루프 적으로 인터럽트 발생으로 인해 변동되는 a값을 상시 반영하도록 만듭니다. OCR0의 값의 변화에 따라 듀티 사이클도 변화합니다. 그리고 중괄호를 닫아 while 문을

제외한 메인 함수를 종료합니다.

완성된 코드가 정상적으로 작동하는지 확인하기 위해 오실로스코프의 화면에서 파동을 확인해야 하기 위해 프로브라는 케이블을 통해 측정합니다. 먼저 프로브의 커넥터를 오실로스코프 본체에 연결한 후, 측정의 정확성을 높이기 위해 그라운드 클립을 보드 아랫면에 회로 부분에 있는 ATmega128의 접지(Ground)와 연결합니다.

프로브 팁을 보드 윗면을 바라보고 OC0이 동작하는 ATmega128 칩의 PB4(Port B의 4번)에 접촉시킵니다. 그렇게 완성된 코드를 통해 오실로스코프에 출력되는 PWM 신호를 측정합니다.

2. Fast PWM mode 또는 Phase Correct PWM mode를 사용하여 1kHZ의 듀티비가 0.6인 펄스 파형을 만드시오.

```
#include<avr/io.h>
#include<avr/interrupt.h>

int a = 127;

int main(void)
{

        DDRB=0xff;      // OC0 출력설정

        PORTB=0xff;      // LED off
        TCCR0=0x6c;     // f=1KHZ, 분주비 64, fast PWM mode
                         // clear OC0 on compare Match, set OC0 at BOTTOM

        TCNT0=0x00;
        OCR0=153;

        while(1);

}       // do nothing
```

fast PWM mode를 사용하고, 주파수를 1kHz로 잡기 위해 분주비를 64로 대입하여 줍니다. 그러면 16MHz/2*64*(1+153)을 계산되어 1kHz에 최대한 가깝게 만듭니다. OCR0은 듀티 사이클을 0.6(60%)로 만들어 주기 위해 듀티 사이클 100% = 256의 약 60% = 153으로 설정합니다. while문을 통해 파형 출력 후 프로그램이 아무것도 하지 않고 대기하도록 합니다. 나머진 추가 실습1과 같은 내용이라 생략했습니다.

5. 측정값

추가실습1은 초기 상태를 지정하고 외부 스위치 동작 시 파형이 변화되는 동작으로 만들었고, 추가실습2는 상태를 설정하고 어떤 값이 출력되는지 테스트하였다.

추가실습1) 외부 인터럽트 발생 시 duty 25%, 50%, 75% 변화

INT0 1번 발생 시	duty 25%로 설정되어 HIGH(=1)인 상태가 전체주기의 1/4 정도로 줄어듭니다. low active LED를 사용하기 때문에 듀티 사이클이 낮을수록 low에 있는 시간이 길어져 LED의 밝기가 증가합니다.
초기 상태	아무런 인터럽트가 발생하지 않은 초기 상태라 duty 50%로 설정되어 있습니다. LED의 밝기가 보통입니다.
INT1 1번 발생 시	duty 75%으로 설정되어 HIGH인 상태가 전체주기의 3/4로 증가하게 됩니다. low active LED를 사용하기 때문에 듀티 사이클이 높을수록 low에 있는 시간이 짧아져 LED의 밝기가 감소합니다.

표 3) 외부 인터럽트를 통한 PWM 기능 테스트 측정값

추가실습2) Fast PWM mode, duty 60%, 1kHZ 신호 설정

상태	오실로스코프 화면에 주파수는 Freq[1] = 976.53Hz로 1kHz에 근접한 양상을 보였고, 듀티비를 60%로 설정하니 파형이 50%일때보다 HIGH인 구간이 길어졌으며 75%보단 짧았습니다. 그리고 Avg-Cyo = 2.9460V로 평균전압이 3에 가깝게 나왔습니다.

표 4) Fast PWM, mode, duty 60%, 1kHZ에 맞춘 PWM 기능 테스트 측정값

6. 실험결과

추가실습1) duty 값이 줄어들수록 밝아지는 LED, 오실로스코프 duty 50%, 25%, 75%



그림 1) 밝기 조절되는 LED



그림 2) 듀티 사이클 변화 (왼쪽-50%, 오른쪽-25%, 아래-75%)

첫 번째 테스트, OC0 함수와 TCNT0 함수를 while문에 같이 넣었더니 오실로스코프 화면에 파형이 출력되지 않았다.

마지막 테스트, TCNT0 함수가 계속 초기화되는 것으로 확인되어 while문에서 제외하고 OC0 함수만 남겨두어 외부 인터럽트에 반응해서 파형을 가변시켜 출력할 수 있도록 만들었다.

정상적으로 PWM 동작을 수행하였다.

추가실습2) Fast PWM mode or PC PWM mode 사용하여 1kHZ의 듀티비가 0.6인 펄스 파형을 만들기



그림 3) Fast PWM, duty 60%, 1kHz 신호

분주비를 64로 입력하여 주파수를 1kHz에 가까운 976.53Hz로 출력시켰다. 5V의 60%에 가까운 2.9560V가 평균전압으로 잡혔고, 관찰되는 펄스폭 또한 듀티 사이클 또한 60%에 가깝게 만들었다.

7. 결과에 대한 논의

아쉬웠던 점은 OCn에 들어갈 주파수를 구할 때 16MHz가 분주비로만 나누어지는 줄 알고 한참을 고민했다는 점이다. 이론 수업 시간 때 교수님이 $f_{OCn} = f_{clkTn}/2*N*(1+OCRn)$라는 공식으로 계산된다고 말씀하신 것을 잊었다는 사실이다. 교재를 훑어 찾아보며 다시금 깨닫게 되었다.

좋았던 점은 저번 실습에서 사용되었던 레지스터의 비트만 조금 바꿔주면 PWM 모드로 바뀌었기 때문에 비교적 다른 실습 때보다을 살려 생각보다 수월하게 진행할 수 있었다는 점이다. 평소 결괏값을 출력하여 확인하던 ATmega128 마이크로 컨트롤러 보드뿐만이 아닌 오실로스코프라는 다른 기기에서도 결괏값을 얻을 수 있다는 사실을 배웠다.

실제로 $f_{OCn} = f_{clkTn}/2*N*(1+OCRn)$로 계산된 결과와 같이 주파수가 나온다는 점과 똑같은 주파수지만 듀티비를 조절한다는 것만으로 펄스 신호의 폭이 감소되거나 증가하는 모습에 코딩과 마이크로 컨트롤러 보드를 통해 파형 또한 조작하고 다룰 수 있다는 것을 알게 되었다.

8. 결론

ATmega128 마이크로 컨트롤러 보드의 PWM 기능을 통해 타이머/카운터0의 레지스터와 OCR0의 레지스터를 비교하여 동작시킬 수 있었다. 그리고 PWM 모드를 선택하고 작성했던 코드가 오실로스코프를 통해 출력되고, 인터럽트를 발생시켜 펄스의 폭을 감소시키거나 증가시키는 과정을 통해 이 실습이 컴퓨터 내에서만 작동할 수 있는 한계가 있는 실습이 아니란 걸 알게 되었다. 코드를 통해 현실에서 역동적으로 움직이는 사물을 조작하는 날을 기대하고 있다.

9. 참고문헌 및 출처

(교재) 김도인 교수님,「실습교재9」, 강원대학교, 전기전자공학과, (2~14p)
(교재) 김도인 교수님 [2024] 마프_Part 09_PWM_ver3, "", (2~30p)
(사진) 이동제와 강민수, 실습 중 찍은 사진, 2024-05-23